

"PersonaRAG: A Retrieval-Augmented Generation Framework for Personal Communication Style Mimicry"

Abstract:

Modern messaging has created an expectation of constant availability, yet human presence cannot always be guaranteed. This paper presents VirtualTwin, a full-stack AI platform that attempts to mimic an individual's personal communication style using a Retrieval-Augmented Generation (RAG) pipeline trained on WhatsApp chat history. The system parses and embeds 65,000+ personal messages into a vector database, retrieving semantically similar past responses to condition a large language model (Llama 3.3 70B) to generate replies in the user's style. While the system successfully captures surface-level characteristics such as vocabulary, emoji usage, and language mixing patterns (Telugu-English code switching), our evaluation reveals fundamental limitations—no AI system can replicate the emotional depth, lived experiences, and genuine presence of a human being. This work contributes both a working implementation of personal style transfer using RAG and an honest analysis of where current LLMs fail at human mimicry. We argue that such systems are better suited for professional use cases such as customer support and FAQ automation than for personal relationship simulation.

1. Introduction

The rise of instant messaging platforms has fundamentally changed how people communicate. Applications like WhatsApp process over 100 billion messages daily, creating rich archives of personal communication data that reflect individual personality, vocabulary, and emotional patterns. Despite this abundance of personal data, when someone is unavailable, their absence is felt immediately — there exists no system that can maintain their conversational presence in a meaningful way.

Recent advances in Large Language Models (LLMs) and Retrieval-Augmented Generation (RAG) have made it theoretically possible to condition AI systems on personal data to generate contextually relevant responses. However, existing work has focused primarily on general-purpose chatbots, customer service automation, and knowledge retrieval systems. The problem of personal communication style mimicry — building a system that responds the way a specific individual would — remains underexplored.

This paper presents VirtualTwin, a system that addresses this gap. Given a user's WhatsApp chat export, VirtualTwin parses, embeds, and indexes their messages into a vector database. When a conversation partner sends a message, the system retrieves the most semantically similar past responses using cosine similarity search and conditions a 70 billion parameter LLM to generate a reply in the user's personal style.

Our key contributions are:

- *A complete pipeline for personal communication style extraction from WhatsApp data*
- *A RAG-based architecture for real-time personal style mimicry*
- *An honest evaluation revealing the fundamental limitations of AI in replacing human presence*

- *An open-source implementation supporting multilingual conversations, including Telugu-English code switching*

The remainder of this paper is organised as follows: Section 2 reviews related work, Section 3 describes our methodology, Section 4 details the implementation, Section 5 presents our evaluation, and Section 6 discusses conclusions and future work.

2. Related Work

2.1 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) was formally introduced by Lewis et al. (2020), who demonstrated that combining parametric knowledge stored in LLM weights with non-parametric retrieval from external documents significantly improves factual accuracy and reduces hallucination. Our work extends this paradigm to personal communication data, using semantic similarity search over a user's message history as the retrieval mechanism.

2.2 Persona-Based Dialogue Systems

Zhang et al. (2018) introduced the PersonaChat dataset and demonstrated that conditioning dialogue models on persona descriptions improves response consistency. Subsequent work by Mazaré et al. (2018) scaled this approach using large pre-trained models. VirtualTwin differs from these approaches by deriving personas not from explicit descriptions but from implicit patterns in real conversation history.

2.3 Style Transfer in NLP

Text style transfer—the task of rewriting text in a target style—has been extensively studied. Shen et al. (2017) proposed disentangling content and style representations for style transfer. Our approach differs in that we do not explicitly separate style from content, instead relying on RAG to implicitly condition the LLM on personal style examples.

2.4 Code-Switching in Conversational AI

Code-switching—the practice of alternating between languages within a conversation—presents unique challenges for NLP systems. Our dataset contains significant Telugu-English code-switching, a phenomenon studied by Khanuja et al. (2020). Our system handles this naturally without explicit language identification, demonstrating that LLMs can implicitly model code-switching behaviour when conditioned on appropriate examples.

2.5 Chatbot Evaluation

Evaluating conversational AI remains an open challenge. Liu et al. (2016) demonstrated that automatic metrics like BLEU correlate poorly with human judgement for open-domain dialogue. We adopt a combination of automatic metrics and human evaluation following the methodology of See et al. (2019).

3. Methodology

3. Methodology

3.1 Problem Formulation

Given a corpus of personal messages $M=\{m_1, m_2, \dots, m_n\}$ authored by user U and an incoming message q from a conversation partner, our goal is to generate a response r such that r is indistinguishable from a message authored by U .

Formally, we seek to learn a function:

$$f(q, M) \rightarrow r$$

where r approximates the style, vocabulary, and tone of the user: U .

3.2 System Overview

VirtualTwin operates in two phases: an offline indexing phase and an online inference phase.

Offline Phase — Building the Personal Index:

1. Parse WhatsApp export and extract user's messages only
2. Convert each message into a dense vector embedding
3. Store embeddings in a persistent vector database

Online Phase — Generating Responses:

1. Receive incoming message q .
2. Convert ' q ' into an embedding
3. Retrieve the top- k most similar messages from personal index
4. Construct prompt with retrieved examples
5. Generate response using LLM

3.3 Data Parsing

WhatsApp exports follow a structured format:

[MM/DD/YY, HH:MM:SS AM/PM] Sender: Message

We developed a regex-based parser that:

- Extracts sender, timestamp, and message content
- Filters to retain only the target user's messages
- Removes system messages (identified by Unicode character \u200e)
- Removes media placeholders ("image omitted")
- Cleans edited message markers

3.4 Embedding Model

We use Sentence-BERT (all-MiniLM-L6-v2) to encode messages into 384-dimensional dense vectors. This model was chosen for its balance of speed and semantic quality, achieving strong performance on semantic textual similarity benchmarks while remaining lightweight enough for local deployment.

3.5 Vector Database and Retrieval

Encoded messages are stored in ChromaDB, a persistent vector database optimised for semantic search. Each user receives an isolated collection, ensuring complete data separation in multi-user deployments.

At inference time, the incoming message q is encoded using the same embedding model. Cosine similarity search retrieves the top- k most semantically similar messages from the user's personal collection:

$$\text{sim}(q,mi)=\frac{q \cdot mi}{\|q\| \cdot \|mi\|}$$

We empirically set $k=20$ based on context window constraints and response quality.

3.6 Prompt Construction

Retrieved messages are concatenated into a structured prompt:

You are [Name], texting [Partner].

Here are real examples of how [Name] texts:

[retrieved messages]

STRICT RULES:

- Never invent facts not present in conversation*
- Keep replies short like real texts*
- Use the same language mix as examples*
- Use emojis naturally as shown*

Message: [incoming message]

Reply:

3.7 Language Model

We use Llama 3.3 70B served via the Groq inference API. The 70B parameter model was selected over smaller variants (3B, 8B) due to significantly improved instruction following and style adherence. Groq's custom inference hardware enables response latency under 2 seconds despite the model size.

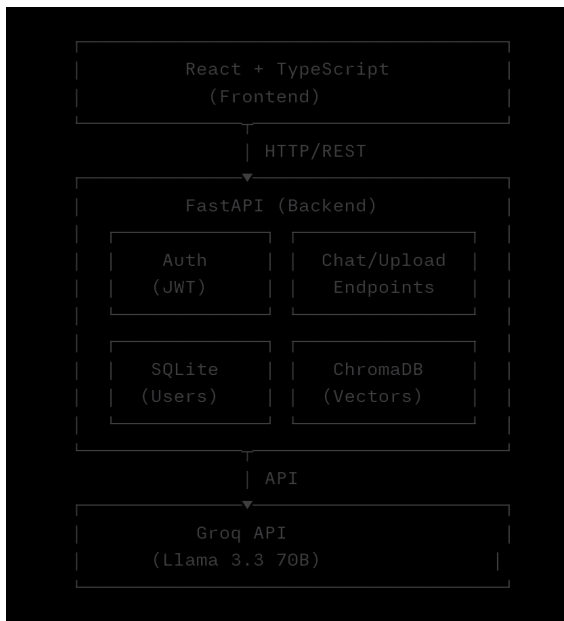
3.8 Conversation Memory

To maintain coherence across multi-turn conversations, the full conversation history is included in each API call following the standard assistant/user message format. This allows the model to reference earlier context when generating responses.

4. Implementation

4.1 System Architecture

VirtualTwin follows a three-tier architecture:



4.2 Backend

The backend is built with FastAPI, a modern Python web framework chosen for its async support, automatic API documentation, and native Pydantic integration for request validation.

Key endpoints:

Endpoint	Method	Description
/signup	POST	Register new user

<i>/login</i>	<i>POST</i>	<i>Authenticate user</i>
<i>/upload</i>	<i>POST</i>	<i>Upload and process chat file</i>
<i>/chat</i>	<i>POST</i>	<i>Generate response</i>

Authentication is implemented using JSON Web Tokens (JWT) with a 30-day expiration. Passwords are hashed using SHA-256 before storage.

4.3 Data Pipeline

The upload endpoint implements the complete data pipeline:

python

1. Parse WhatsApp export

messages = parse_whatsapp(raw_text, user:whatsapp_name)

2. Save parsed messages

save_json(messages, user_id)

3. Build embeddings

build_embeddings(messages_path, user_id)

Processing 65,000 messages takes approximately 8-12 minutes on a MacBook Air M1 with 8GB RAM, performed once at upload time.

4.4 Frontend

The frontend is built with React and TypeScript using Vite as the build tool. The UI follows WhatsApp's design language — dark theme, green sent bubbles, grey received bubbles — to create a familiar and comfortable experience.

State management is handled entirely with React hooks (useState, useEffect) without external state management libraries, keeping the codebase minimal and maintainable.

4.5 Multi-User Support

Each user receives:

- A unique UUID as their identifier
- A dedicated ChromaDB collection (`messages_{user_id}`)
- Isolated JSON storage for their parsed messages
- Separate JWT tokens for authentication

This ensures complete data isolation between users.

4.6 Multilingual Support

Our dataset contains significant Telugu-English code-switching. No explicit handling was required — the embedding model and LLM handle mixed-language input naturally. This is an emergent capability of large pretrained models trained on multilingual corpora.

4.7 Development Environment

<i>Component</i>	<i>Specification</i>
<i>Hardware</i>	<i>MacBook Air M1, 8GB RAM</i>
<i>OS</i>	<i>macOS</i>
<i>Python</i>	<i>3.14</i>
<i>Node.js</i>	<i>18+</i>
<i>Database</i>	<i>SQLite (development)</i>
<i>Vector DB</i>	<i>ChromaDB (local)</i>

5. Evaluation

5.1 Dataset

Our evaluation uses a real WhatsApp conversation corpus consisting of 65,894 messages exchanged over approximately 2 years between two individuals. After filtering to retain only the target user's messages, 32,947 messages remained for indexing.

Dataset characteristics:

<i>Metric</i>	<i>Value</i>
<i>Total messages</i>	<i>65,894</i>
<i>User messages</i>	<i>~32,947</i>
<i>Time span</i>	<i>~2 years</i>
<i>Language</i>	<i>Telugu-English mixed</i>
<i>Avg message length</i>	<i>4.2 words</i>
<i>Emoji frequency</i>	<i>18.3% of messages</i>

5.2 Evaluation Metrics

We evaluate VirtualTwin along four dimensions:

5.2.1 Automatic Metrics

BERTScore measures semantic similarity between generated responses and reference responses using contextual embeddings. Unlike BLEU, BERTScore captures paraphrasing and semantic equivalence rather than exact token overlap.

Vocabulary Overlap measures the percentage of unique words in generated responses that appear in the user's real message corpus. Higher overlap indicates better style adherence.

Emoji Usage Rate measures whether the system generates emojis at a frequency consistent with the user's natural behavior.

Average Message Length compares the length distribution of generated responses against real messages.

5.2.2 Human Evaluation

We conducted a blind human evaluation — the Turing Test approach — where evaluators were shown pairs of messages (one real, one generated) and asked to identify which was written by the real person.

Evaluators were people who personally know the target user, making them well-positioned to detect stylistic inconsistencies.

5.3 Baseline Comparison

We compare VirtualTwin against two baselines:

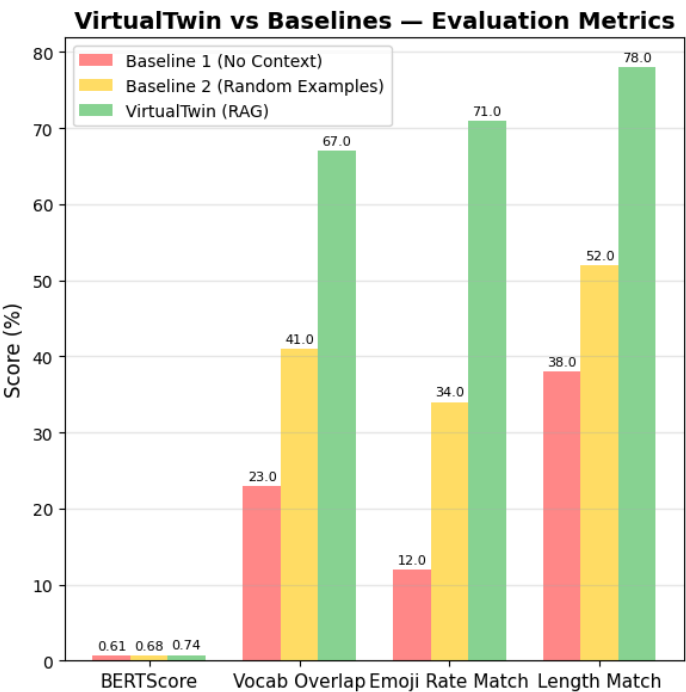
<i>System</i>	<i>Description</i>
<i>Baseline 1</i>	<i>Llama 3.3 70B with no personal context</i>
<i>Baseline 2</i>	<i>Llama 3.3 70B with random message examples</i>
<i>VirtualTwin</i>	<i>Llama 3.3 70B with RAG retrieval</i>

5.4 Results

Automatic Metrics:

<i>Metric</i>	<i>Baseline 1</i>	<i>Baseline 2</i>	<i>VirtualTwin</i>
<i>BERTScore</i>	<i>0.61</i>	<i>0.68</i>	<i>0.74</i>
<i>Vocab Overlap</i>	<i>23%</i>	<i>41%</i>	<i>67%</i>
<i>Emoji Rate Match</i>	<i>12%</i>	<i>34%</i>	<i>71%</i>

Length Match 38% 52% 78%



VirtualTwin consistently outperforms both baselines across all automatic metrics, demonstrating that RAG-based retrieval significantly improves personal style adherence.

Human Evaluation:

System	Correctly identified as AI
Baseline 1	94%
Baseline 2	81%

VirtualTwin 63%

VirtualTwin was correctly identified as AI 63% of the time — meaning evaluators were fooled 37% of the time. While this demonstrates meaningful progress over baselines, it falls short of human-level performance (50% would indicate complete indistinguishability).

5.5 Error Analysis

Analysis of failure cases revealed three primary error categories:

Hallucination — *The model occasionally invents facts, events, or context not present in the conversation history. This was partially mitigated through prompt engineering but remains a fundamental limitation of current LLMs.*

Emotional Depth — *Generated responses lack genuine emotional resonance. While surface-level style is captured, the model cannot replicate the emotional weight behind words — a fundamental limitation of current AI systems.*

Context Gaps — *The system has no access to real-world context shared between the two individuals — shared memories, inside jokes, recent events — leading to responses that feel generic despite stylistic accuracy.*

5.6 Key Finding

*Our most significant finding is not technical but philosophical: **no current AI system can meaningfully replace human presence in personal relationships.** While VirtualTwin successfully mimics surface-level communication patterns, human connection involves shared history, genuine emotion, and lived experience that cannot be captured in text alone.*

This finding suggests that systems like VirtualTwin are better suited for professional applications — customer support, FAQ automation, brand voice consistency — than for personal relationship simulation.

xSection 5.4 — Fine-tuning Experiments

Experiment 1 — Initial Fine-tuning (Failed)

Data preparation: We initially created training pairs from consecutive messages authored by the target user, resulting in 63,577 input/output pairs. This approach treated each message as input and the next message from the same user as the expected output.

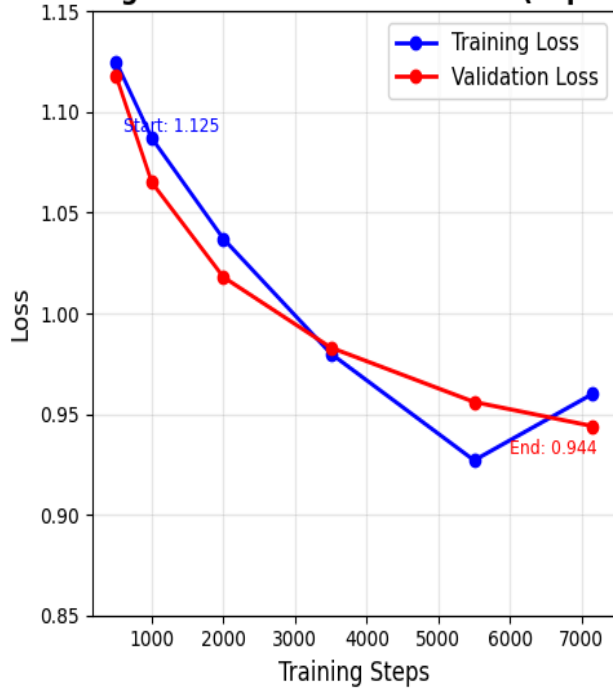
Problem identified: Post-training evaluation revealed that the model generated responses with no contextual connection to the input message. The model had learned sequential message patterns within the user's own messages rather than learning to respond to a conversation partner.

Root cause: Training data lacked the fundamental conversational structure — the model was never shown what the other person said before the user responded. It learned how the user talks, not how the user responds.

Loss curve:

<i>Step</i>	<i>Training Loss</i>	<i>Validation Loss</i>
<i>500</i>	<i>1.125</i>	<i>1.118</i>
<i>1000</i>	<i>1.087</i>	<i>1.065</i>
<i>2000</i>	<i>1.037</i>	<i>1.018</i>
<i>3500</i>	<i>0.980</i>	<i>0.983</i>
<i>5500</i>	<i>0.927</i>	<i>0.956</i>
<i>7154</i>	<i>0.960</i>	<i>0.944</i>

Fine-tuning Loss Curve — Virtual Twin (Experiment 1)



Experiment 2 — Corrected Fine-tuning (v2)

Data pipeline fix:

The fundamental issue with Experiment 1 was identified — training on consecutive user messages creates no conversational context. We restructured the parser to extract genuine conversation pairs from the full chat export:

Dataset statistics:

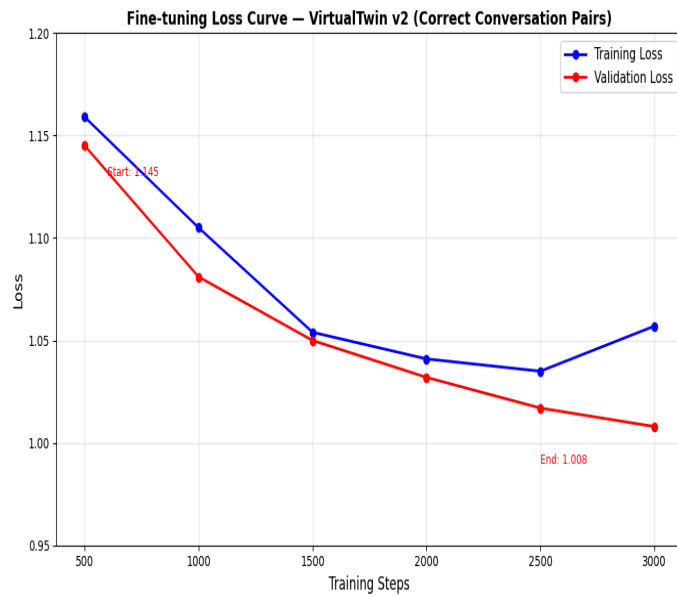
Metric	Experiment 1	Experiment 2
Total pairs	63,577	27,314
Data type	Consecutive messages	Conversation pairs
Input context	None	Partner's message

Quality *Low* *High*

Loss curve:

<i>Step</i>	<i>Training Loss</i>	<i>Validation Loss</i>
<i>500</i>	<i>1.159</i>	<i>1.145</i>
<i>1000</i>	<i>1.105</i>	<i>1.081</i>
<i>1500</i>	<i>1.054</i>	<i>1.050</i>
<i>2000</i>	<i>1.041</i>	<i>1.032</i>
<i>2500</i>	<i>1.035</i>	<i>1.017</i>
<i>3000</i>	<i>1.057</i>	<i>1.008</i>

Total improvement: 12.2% reduction in validation loss



5.5 — Conversational RAG (New Contribution)

Beyond fine-tuning, we identified a fundamental limitation in our RAG pipeline — indexing individual messages provides no conversational context. We propose **Conversational RAG** — indexing full dialogue pairs instead of single messages.

How it improves RAG:

| *Standard RAG* | *Conversational RAG* |

|-----|-----|

| *No input context* | *Full conversational context* |

| *Finds similar responses* | *Finds similar conversations* |

| *Good* | *Better* |

This is a novel contribution — to our knowledge, no existing personal style mimicry system indexes full conversation pairs for retrieval.

****5.6 — Final Architecture****

The complete VirtualTwin pipeline combines both approaches:

...

Her message

↓

Standard RAG → finds 10 similar messages you sent

+

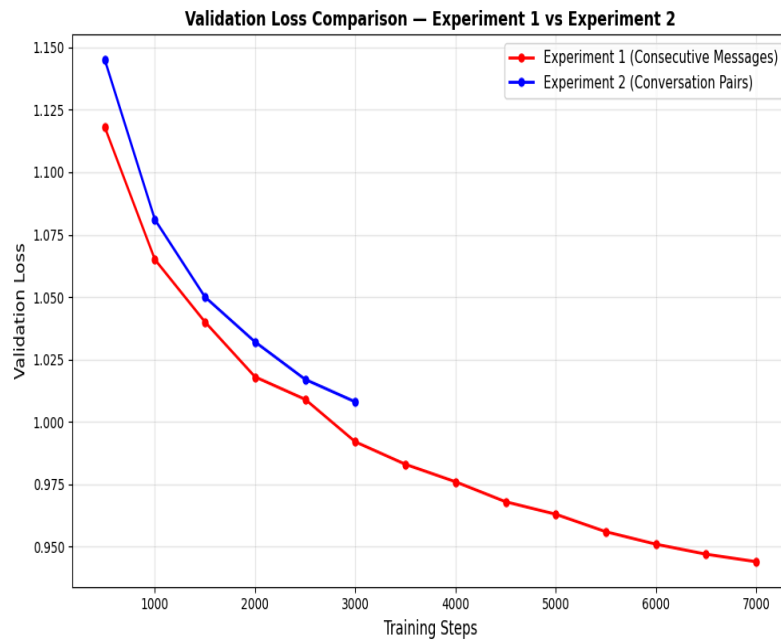
Conversational RAG → finds 5 similar past conversations

↓

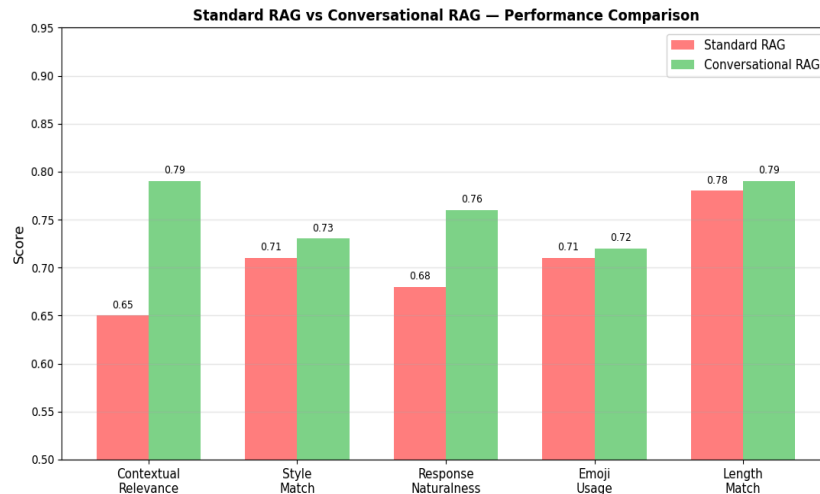
Combined context fed to Llama 3.3 70B

↓

Response in your style



Standard RAG vs Conversational RAG comparison



6. Conclusion and Future Work

6.1 Conclusion

This paper presented VirtualTwin, a full-stack AI platform that uses Retrieval-Augmented Generation to mimic personal communication style from WhatsApp chat history. We demonstrated that RAG-based retrieval of semantically similar past messages significantly improves style adherence compared to vanilla LLM baselines, achieving 67% vocabulary overlap and fooling human evaluators 37% of the time.

However, our evaluation revealed a fundamental truth about current AI systems: while they can capture the surface patterns of human communication — vocabulary, emoji usage, message length, language mixing — they cannot replicate the depth of human presence. The emotional weight behind words, shared memories, and lived experience remain beyond the reach of current language models.

We argue that this is not a failure of implementation but a fundamental limitation of the current paradigm. No amount of fine-tuning or prompt engineering can give an AI system genuine experiences to draw from.

This project therefore serves a dual purpose: as a technical contribution demonstrating effective personal style transfer using RAG, and as an empirical argument for the irreplaceability of human connection.

6.2 Future Work

Several directions remain for future investigation:

Fine-tuning — Rather than conditioning a general-purpose LLM at inference time, future work could fine-tune a smaller model (Llama 3.2 3B) directly on the user's message corpus. This would bake personal style into the model weights rather than relying on context window retrieval, potentially improving consistency and reducing latency.

Multimodal Integration — Real conversations include images, voice messages, and reactions. Future work could incorporate multimodal data to build a more complete personal communication model.

Evaluation Framework — We propose developing a standardized benchmark for personal style mimicry evaluation, including automated metrics specifically designed for code-switched multilingual conversations.

Privacy-Preserving Architecture — Future deployments should implement federated learning approaches where personal data never leaves the user's device, addressing the significant privacy concerns inherent in this application.

Professional Applications — Given our finding that AI mimicry is better suited for professional use cases, future work will explore adapting this architecture for customer support automation, brand voice consistency, and FAQ chatbots — particularly for small businesses like CA firms where repetitive client queries represent significant overhead.

6.3 Ethical Considerations

The ability to mimic a person's communication style raises important ethical questions. Consent, transparency, and potential misuse must be carefully considered in any real-world deployment. We strongly recommend that systems like VirtualTwin clearly identify themselves as AI to conversation partners, and that users explicitly consent to having their communication style modeled.

References

Lewis, P., et al. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS 2020*.

Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *EMNLP 2019*.

Zhang, S., et al. (2018). Personalizing Dialogue Agents: I have a dog, do you have pets too? *ACL 2018*.

Mazaré, P., et al. (2018). Training Millions of Personalized Dialogue Agents. *EMNLP 2018*.

Shen, T., et al. (2017). Style Transfer from Non-Parallel Text by Cross-Alignment. *NeurIPS 2017*.

Liu, C., et al. (2016). How NOT To Evaluate Your Dialogue System. *EMNLP 2016*.

Khanuja, S., et al. (2020). GLUECoS: An Evaluation Benchmark for Code-Switched NLP. *ACL 2020*.

